06 - Logistic Regression

Pattern Recognition and Machine Learning, Lab 6

Michal Neoral
neoramic@fel.cvut.cz

30 October 2026

Recap

Where are we?

Lab 2: Bayes

- $p(x \mid k), p(k)$ **known**
- optimal strategy $q^*(x)$

We know **everything**.

Labs 4 and 5: estimate

- $p(x \mid k)$ **unknown**
- estimate it: Parzen, ML, MAP, Bayes
- then plug into Bayes

Generative: a model of how x looks.

Today

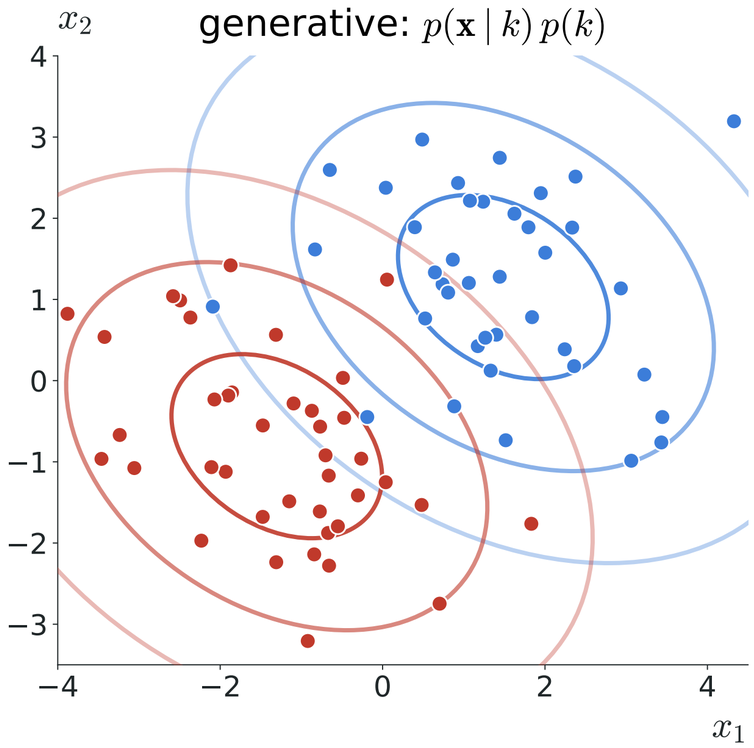
- we need only $p(k \mid x)$
- estimate it **directly**

Discriminative: a model of the decision.

today

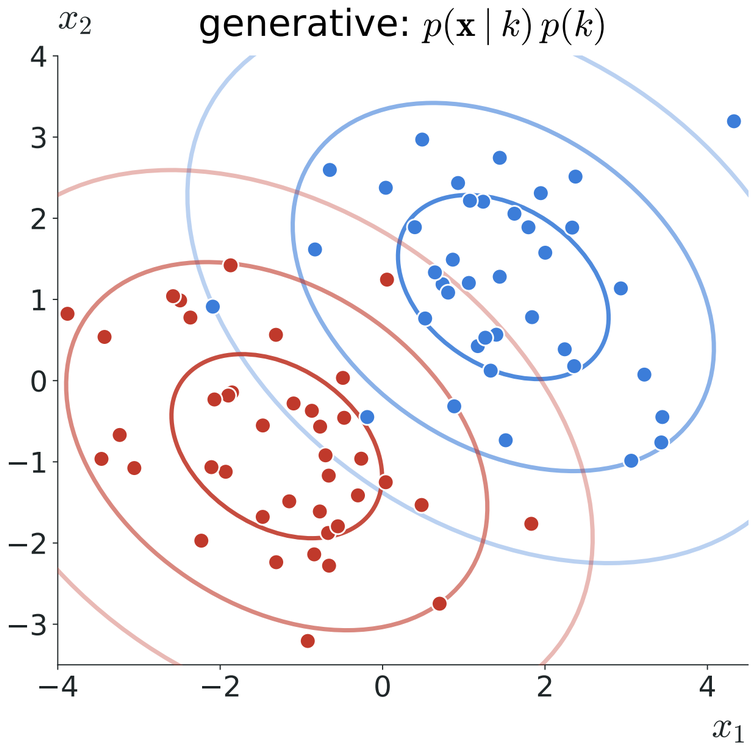
! To decide, we do not need to know how the data look. We need $p(k \mid x)$, nothing more.

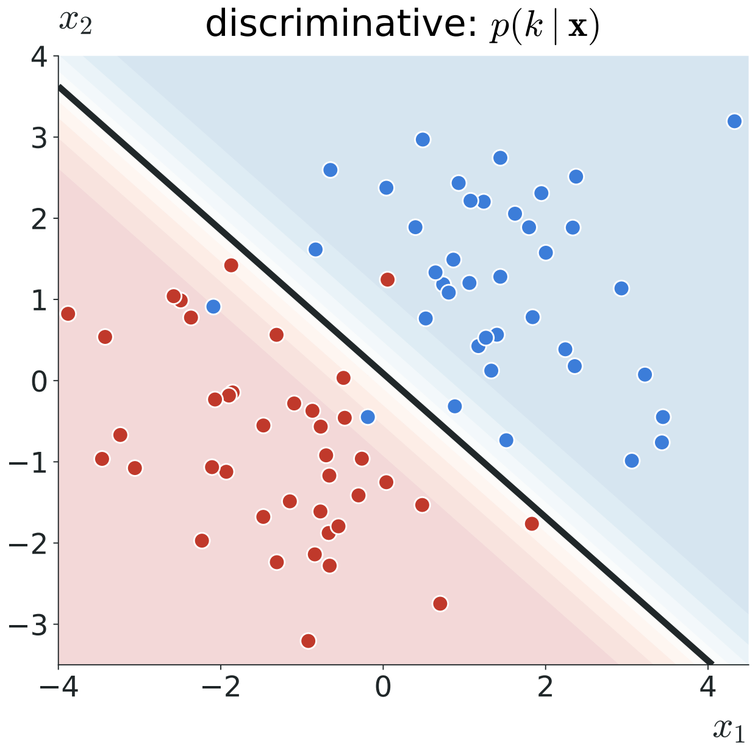
Two ways to the posterior



Generative: model $p(x \mid k)\, p(k)$, get $p(k \mid x)$ by Bayes. Can **generate** new data.

Two ways to the posterior





Discriminative: model only $p(k \mid x)$. Simpler, but **no model of the data**.

From Bayes to log odds

Bayesian strategy, 0/1 loss, two classes

$$q^*(x) = \arg\max_k p(k \mid x)$$

$$q^*(x) = 1 \quad \text{if} \quad p(1 \mid x) > p(2 \mid x) \;\Leftrightarrow\; \frac{p(1 \mid x)}{p(2 \mid x)} > 1 \;\Leftrightarrow\; \underbrace{\ln \frac{p(1 \mid x)}{p(2 \mid x)}}_{a(x) \text{ log odds}} > 0$$

? Have you seen this ratio before?

! **Not** the likelihood ratio $\frac{p(x|1)}{p(x|2)}$ **from minimax. This is the ratio of the posteriors, the odds.**

The simplest choice: a linear log odds

Model the log odds **directly**, as a linear function of x:

$$a(x) = \ln \frac{p(1 \mid x)}{p(2 \mid x)} = w_0 + \mathbf{w}_1^\top x = \mathbf{w}^\top \mathbf{x}$$

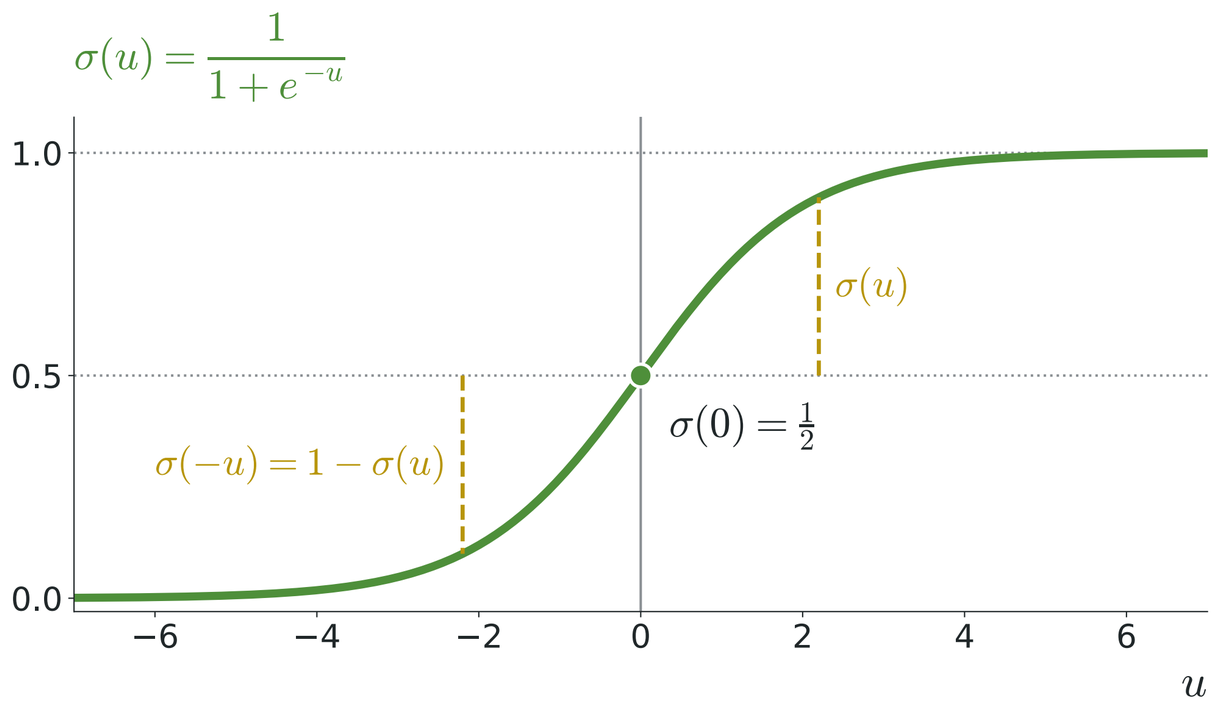
The bias is hidden in the vector: $\mathbf{x} = [1, x]$, $\mathbf{w} = [w_0, \mathbf{w}_1]$.

1. Express $p(1 \mid x)$ and $p(2 \mid x)$ using $a(x) = \mathbf{w}^\top \mathbf{x}$.
2. Show that $1 - \sigma(u) = \sigma(-u)$ for $\sigma(u) = \frac{1}{1+e^{-u}}$.

More problems like this: exercise book on CourseWare.

For many problems the log odds **is** linear: e.g. two normal classes with the **same covariance** (later).

The logistic sigmoid



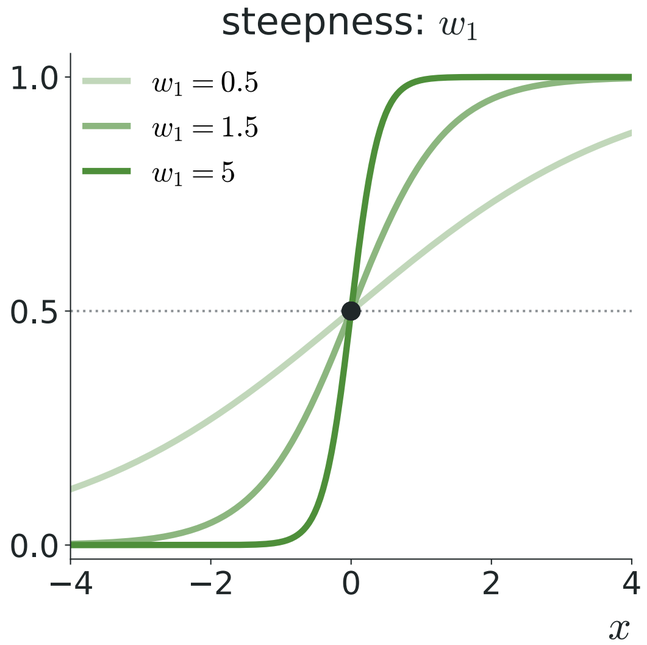
$$p(1 \mid x) = \sigma(\mathbf{w}^\top \mathbf{x})$$
$$p(2 \mid x) = \sigma(-\mathbf{w}^\top \mathbf{x})$$

- a **probability** in $(0, 1)$
- $\sigma(0) = \frac{1}{2}$: the **boundary**
- $\sigma(-u) = 1 - \sigma(u)$
- $\sigma' = \sigma\,(1 - \sigma)$

? Does it look familiar?

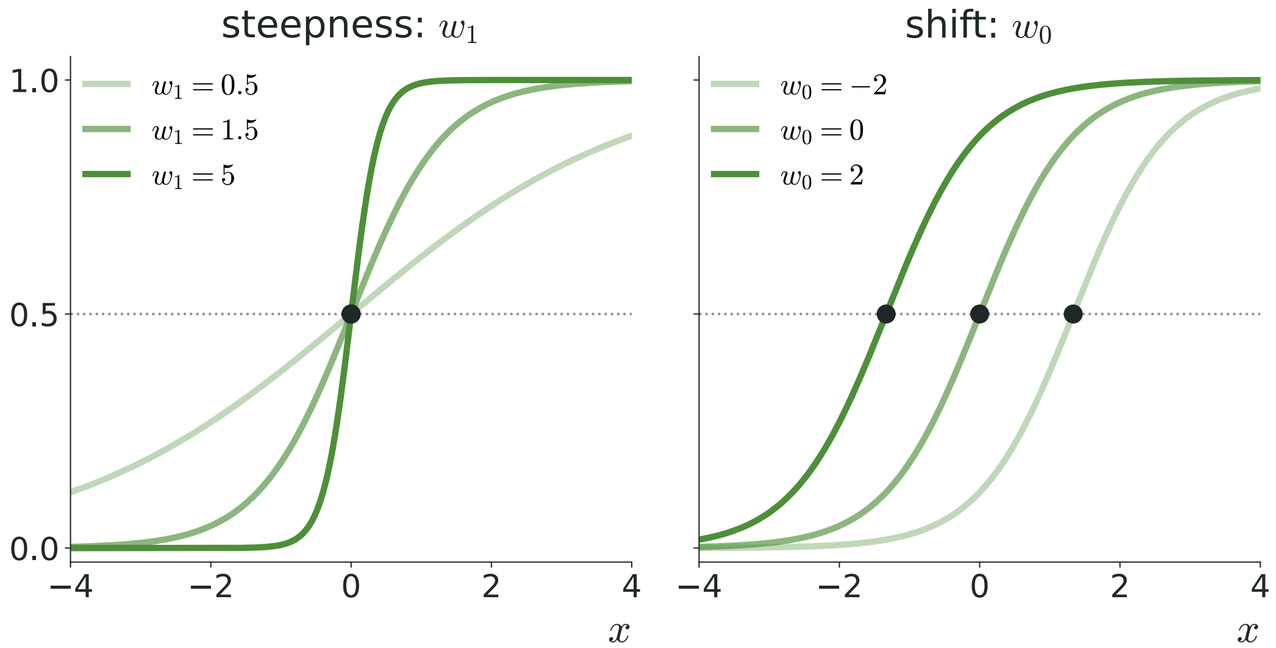
$\tanh u = 2\,\sigma(2u) - 1$, a normal CDF (Lab 2)

What w_0 and w_1 do (1D)



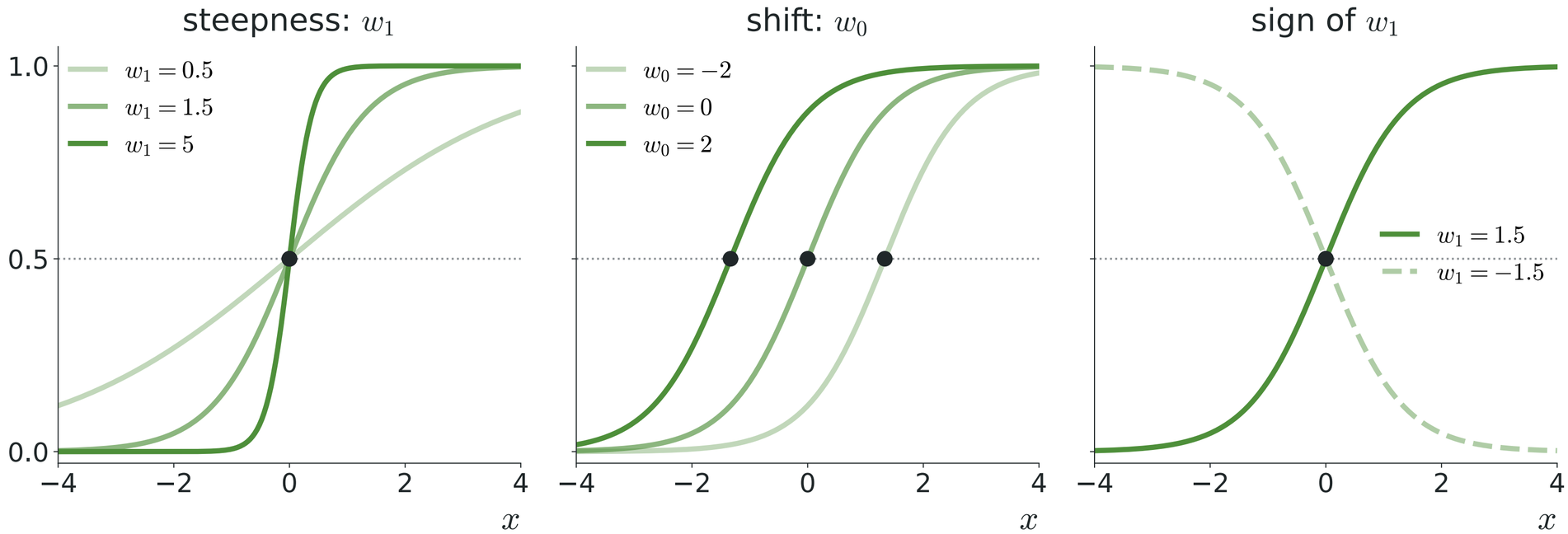
$p(+1 \mid x) = \sigma(w_0 + w_1 x)$: larger $|w_1|$, **steeper**, more confident.

What w_0 and w_1 do (1D)



w_0 **shifts** the curve: $p = \frac{1}{2}$ at $x = -w_0/w_1$.

What w_0 and w_1 do (1D)



The **sign** of w_1: which class is on which side.

Logistic regression: try it

w_0 0

w_1 -1

- [x] p(+1 | x)
- [x] fit of each sample
- [x] ML solution

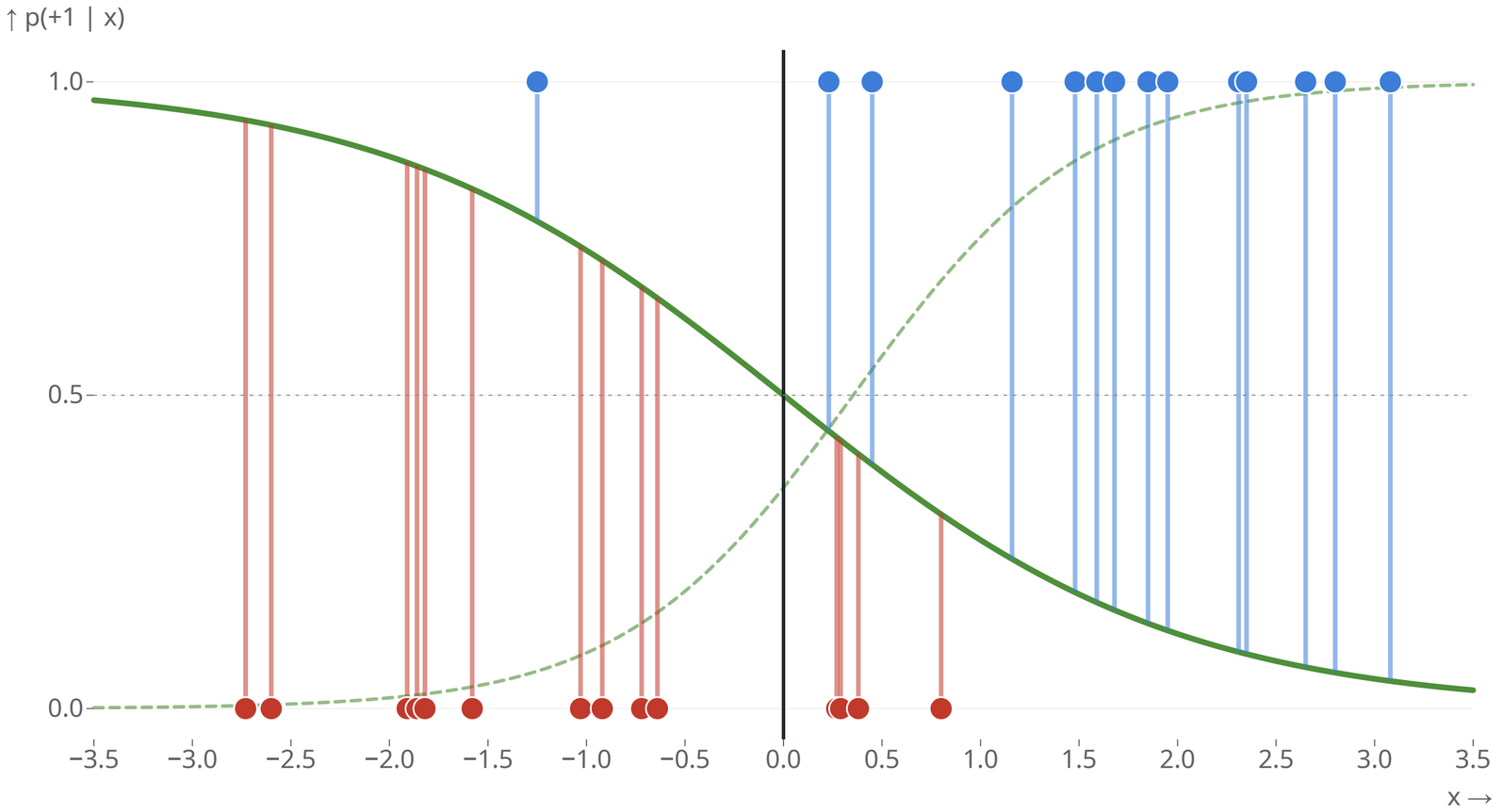
N = 28 samples

E(w) = **1.658**

errors: **23**

ML: w_0 = -0.61, w_1 = 1.72

E(w) = 0.314, errors: 4



The threshold in 1D

$$p(+1 \mid x) = \sigma(w_0 + w_1 x)$$

Labels $k \in \{+1, -1\}$ from now on, as in the assignment.

1. For which x is $p(+1 \mid x) = p(-1 \mid x) = \frac{1}{2}$?
2. $\mathbf{w} = (1.5, -0.7)$: the threshold? Which side is $+1$?
3. Write the strategy $q(x)$.

More problems like this: exercise book on CourseWare.

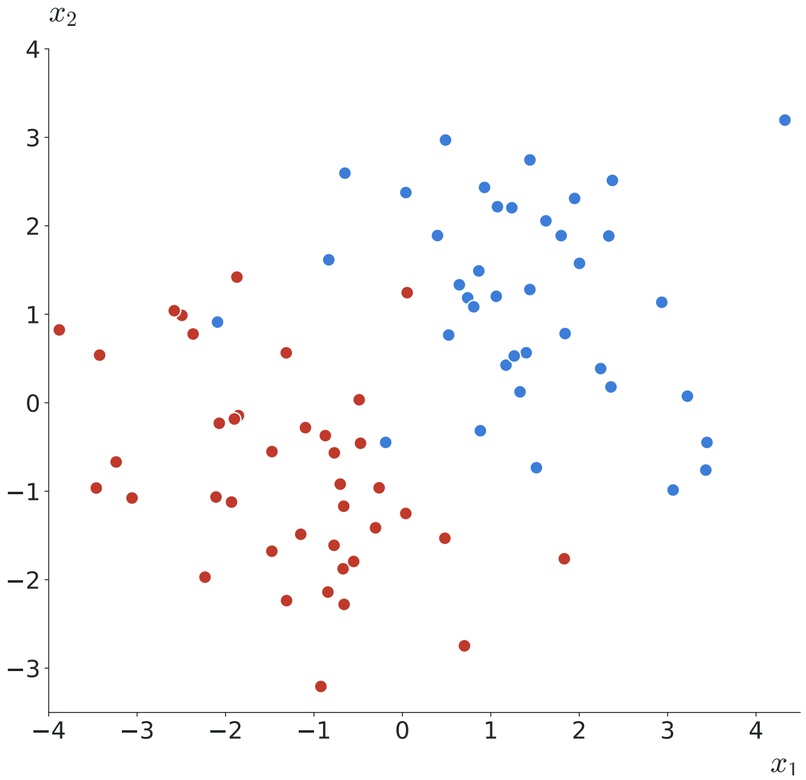
Derivation: the threshold in 1D

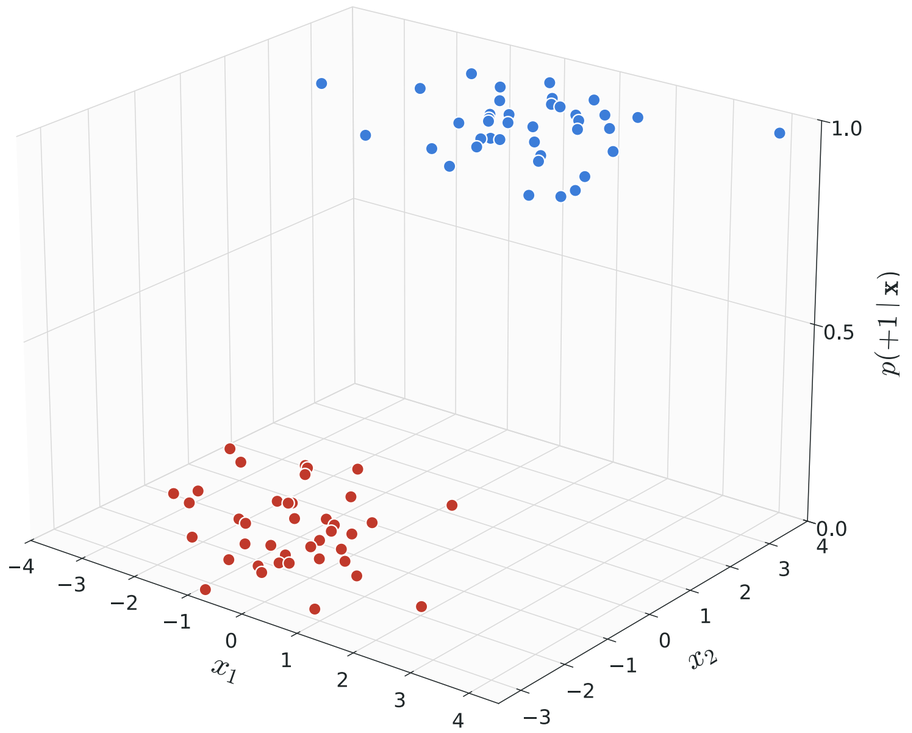
$$\sigma(w_0 + w_1 x) = \tfrac{1}{2} \;\Leftrightarrow\; w_0 + w_1 x = 0 \;\Leftrightarrow\; \boxed{x^* = -\frac{w_0}{w_1}}$$

$$\mathbf{w} = (1.5, -0.7): \qquad x^* = -\frac{1.5}{-0.7} \approx 2.14, \qquad w_1 < 0 \;\Rightarrow\; p(+1 \mid x) \text{ decreases}$$

$$q(x) = \begin{cases} +1 & \text{if } x < 2.14 \\ -1 & \text{if } x \geq 2.14 \end{cases}$$

More dimensions: a linear decision boundary

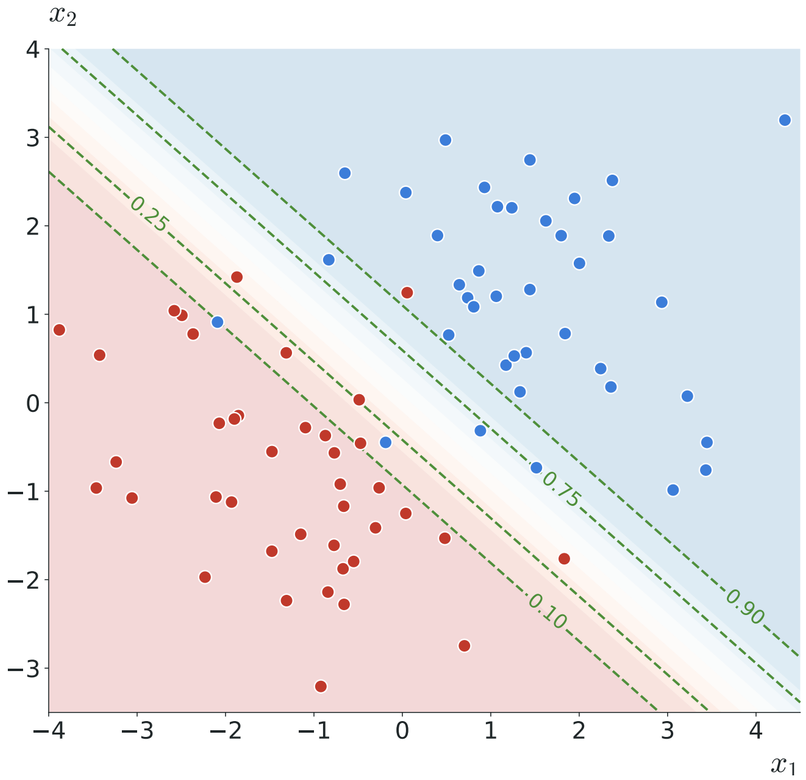


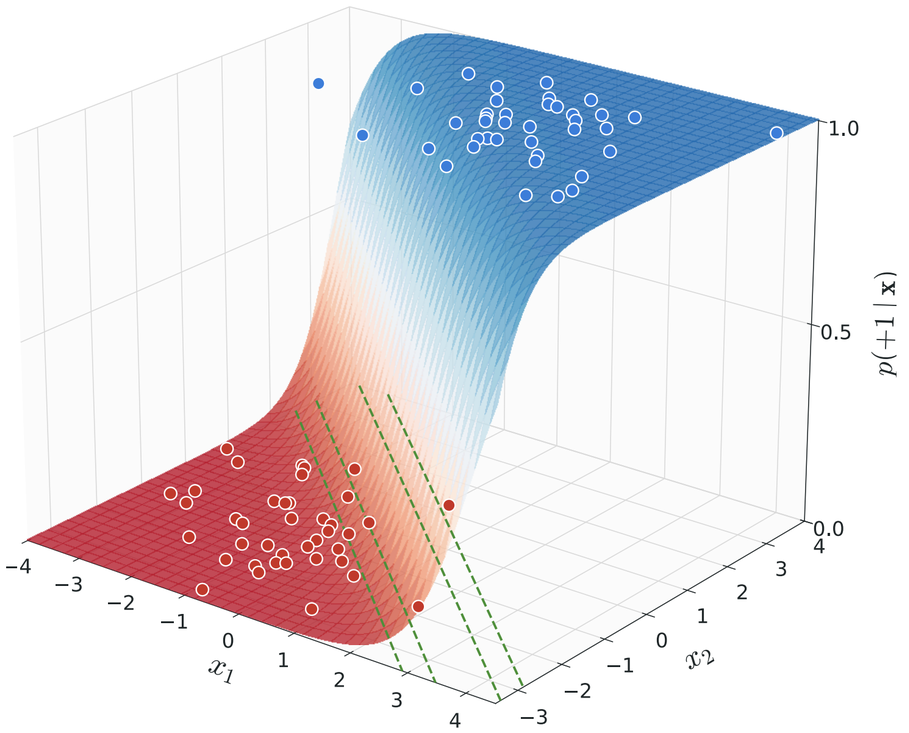


$x \in \mathbb{R}^2, p(+1 \mid x) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$: a **surface** over the plane.

> **!** Logistic regression is a **linear classifier**: it can only cut the space by a hyperplane.

More dimensions: a linear decision boundary

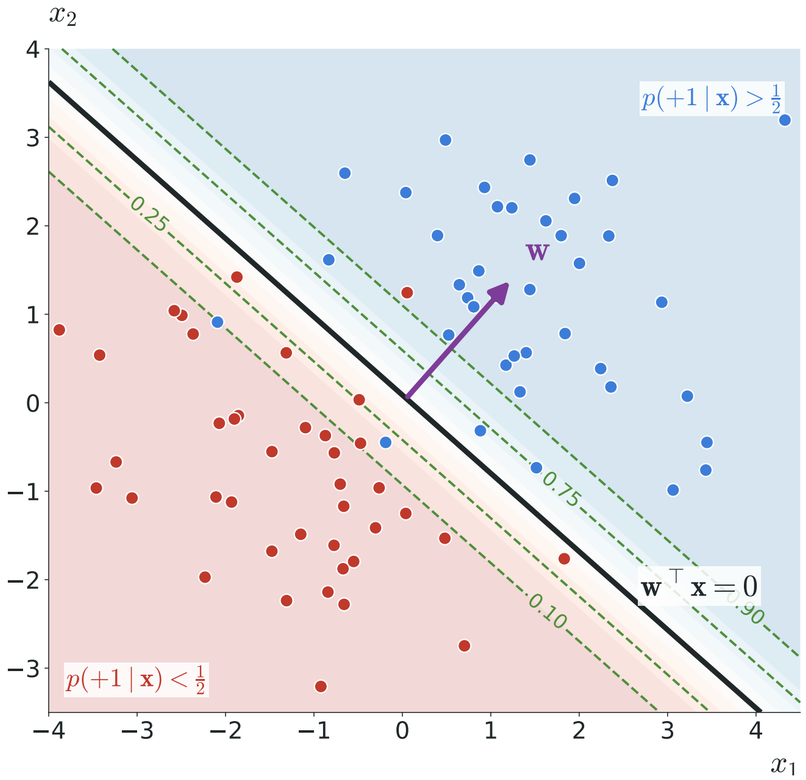


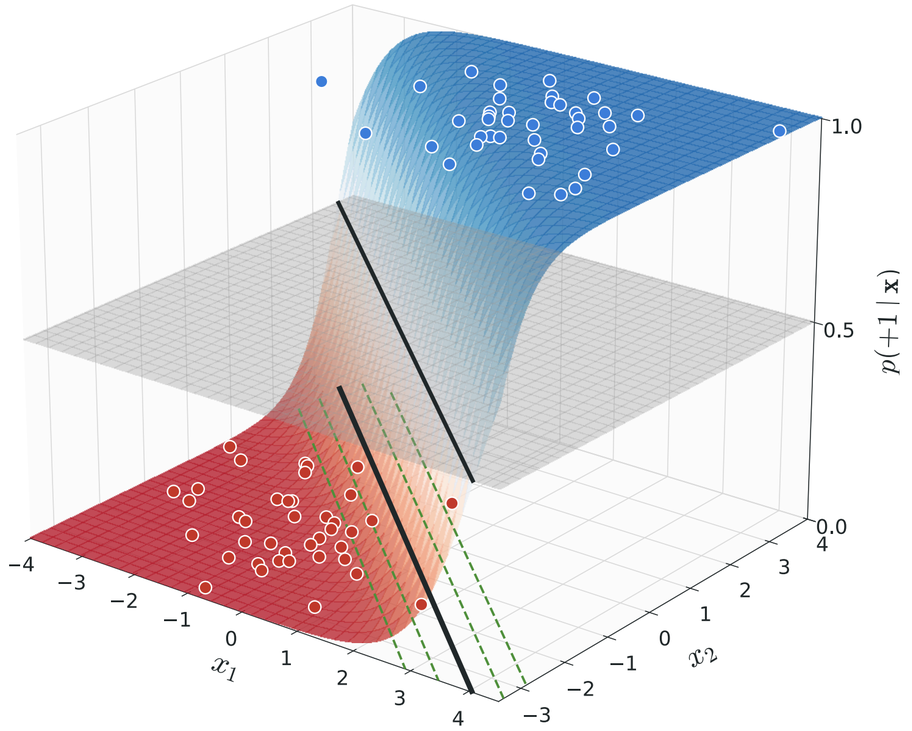


A sigmoid along $\mathbf{w}$, **constant** across it: the contours are **parallel lines**.

> **!** Logistic regression is a **linear classifier**: it can only cut the space by a hyperplane.

More dimensions: a linear decision boundary





Boundary $\mathbf{w}^\top \mathbf{x} = 0$: the surface cut at $\frac{1}{2}$, a **line** ($\mathbb{R}^D$: a **hyperplane**) $\perp \mathbf{w}$.

Logistic regression is a **linear classifier**: it can only cut the space by a hyperplane.

Learning w

Labels ± 1: one formula for both classes

$$p(+1 \mid x) = \sigma(\mathbf{w}^\top \mathbf{x})$$
$$p(-1 \mid x) = \sigma(-\mathbf{w}^\top \mathbf{x})$$

$$\Downarrow$$

$$\boxed{p(k \mid x) = \sigma(k\,\mathbf{w}^\top \mathbf{x})}$$

Training set:

$$T = \{(x_1, k_1), \ldots, (x_N, k_N)\}$$

$$k_i \in \{+1, -1\}$$

? How to find $\mathbf{w}$? We have seen it last week.

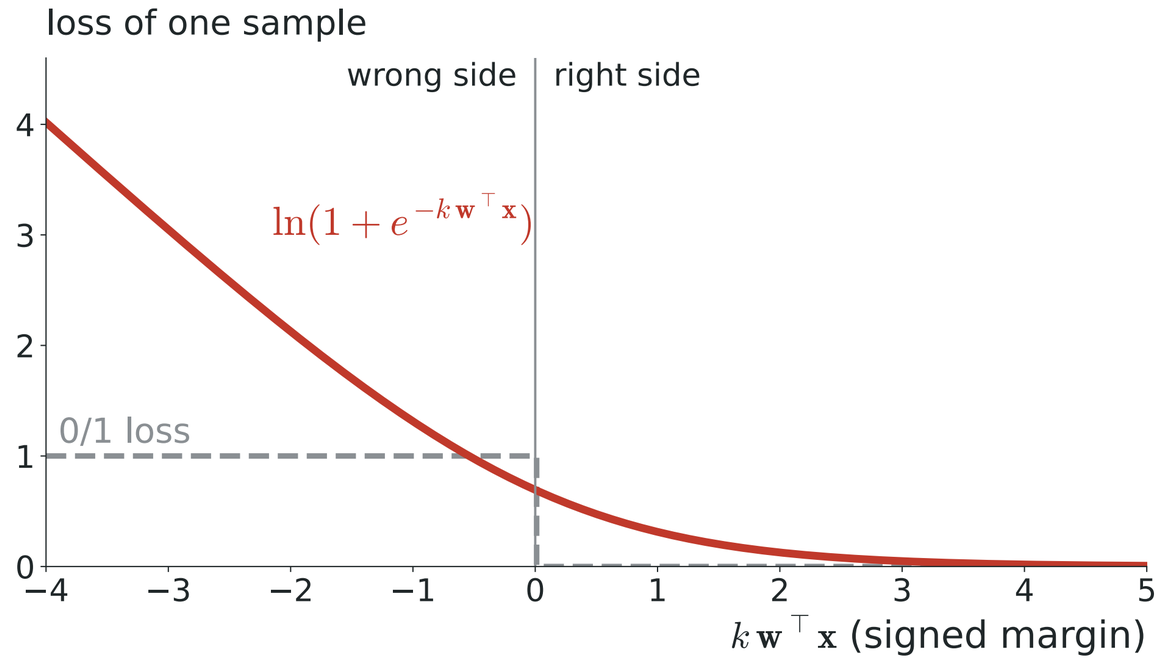
Maximum likelihood for $\mathbf{w}$

$$\ell(\mathbf{w}) = \sum_{i=1}^{N} \ln p(x_i, k_i \mid \mathbf{w}) = \sum_{i=1}^{N} \ln p(k_i \mid x_i, \mathbf{w}) + \underbrace{\sum_{i=1}^{N} \ln p(x_i)}_{\text{does not depend on } \mathbf{w}}$$

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \sum_{i=1}^{N} \ln \sigma(k_i \, \mathbf{w}^\top \mathbf{x}_i) = \arg\min_{\mathbf{w}} \underbrace{\frac{1}{N} \sum_{i=1}^{N} \ln \left(1 + e^{-k_i \, \mathbf{w}^\top \mathbf{x}_i}\right)}_{E(\mathbf{w})}$$

Result: $E(\mathbf{w})$ is **convex**: one minimum, but **no closed form**. We need a numerical method.

Cross-entropy loss



Loss of one sample:

$$-\ln p(k_i \mid x_i)$$

- right side, confident: loss ≈ 0
- on the boundary: $\ln 2$
- wrong side: grows **linearly**

A smooth, convex replacement of the 0/1 loss. We can take its **gradient**.

Cross-entropy, entropy, KL divergence

True distribution p, model q:

$$H(p,q) = -\sum_k p(k)\ln q(k) = \underbrace{-\sum_k p(k)\ln p(k)}_{\text{entropy } H(p)} + \underbrace{\sum_k p(k)\ln\frac{p(k)}{q(k)}}_{D_{\mathrm{KL}}(p\,\|\,q)\geq 0}$$

One sample, true label k_i: p is **one-hot**, $H(p) = 0$

$$H(p,q) = -\ln q(k_i) = \ln\left(1 + e^{-k_i\,\mathbf{w}^\top \mathbf{x}_i}\right)$$

Minimizing cross-entropy = minimizing KL = **maximizing likelihood**.

The gradient of $E(\mathbf{w})$

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \ln\left(1 + e^{-k_i \, \mathbf{w}^\top \mathbf{x}_i}\right)$$

1. Compute the gradient $\nabla E(\mathbf{w})$.
2. Which samples push the most?

More problems like this: exercise book

on CourseWare.

Derivation: the gradient

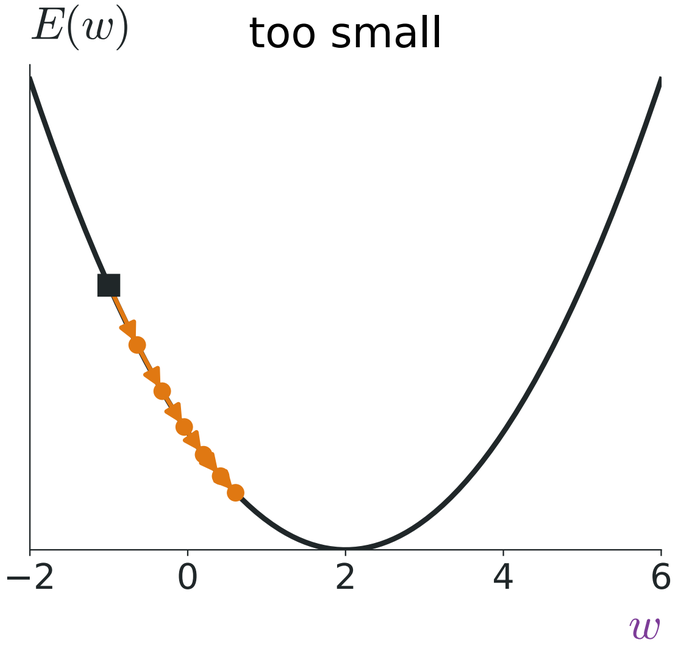
$$\nabla E(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \frac{e^{-k_i\,\mathbf{w}^\top \mathbf{x}_i}}{1+e^{-k_i\,\mathbf{w}^\top \mathbf{x}_i}}\,(-k_i\,\mathbf{x}_i) = -\frac{1}{N}\sum_{i=1}^{N} \sigma(-k_i\,\mathbf{w}^\top \mathbf{x}_i)\,k_i\,\mathbf{x}_i$$

$$\boxed{\nabla E(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N} \big(1 - p(k_i \mid x_i)\big)\,k_i\,\mathbf{x}_i}$$

badly classified samples push the most

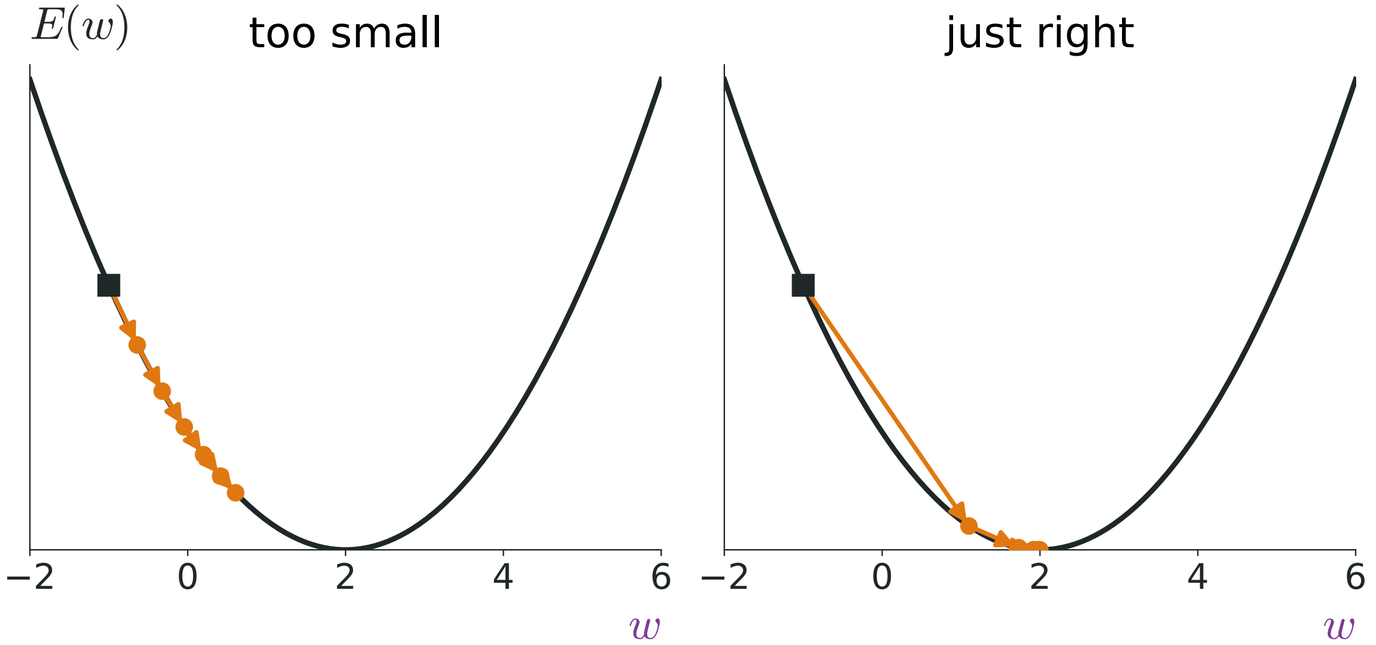
Hessian: $H(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} p(+1 \mid x_i)\,p(-1 \mid x_i)\,\mathbf{x}_i\mathbf{x}_i^\top \succeq 0 \quad \Rightarrow \quad E$ is convex

Gradient descent: the step size



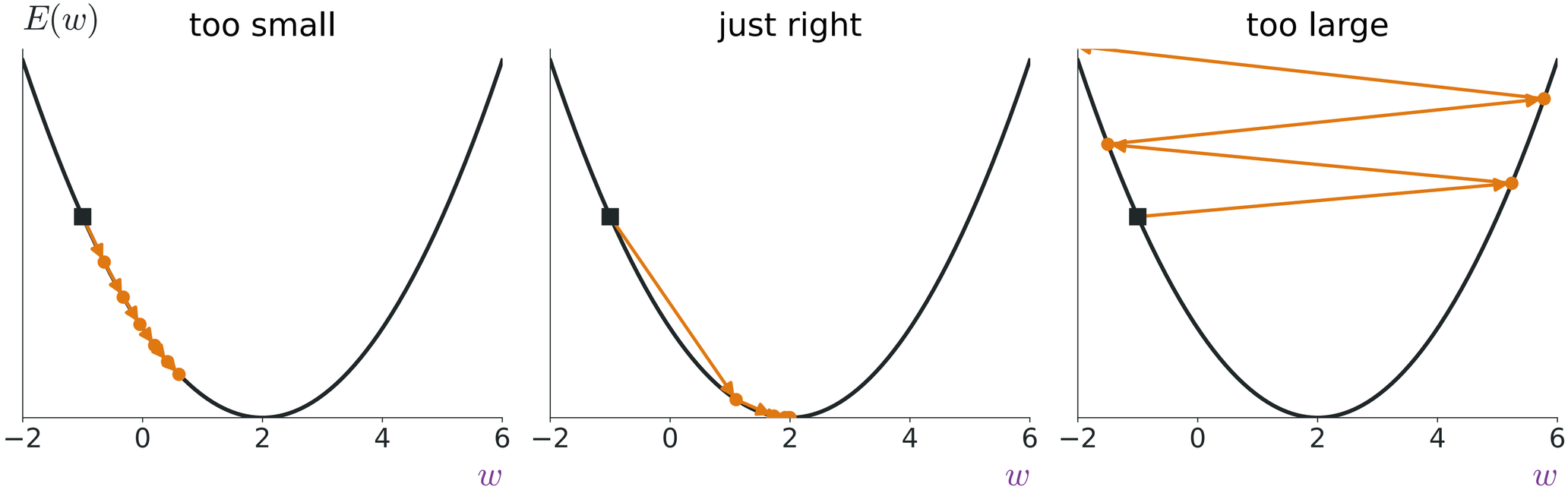
$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E(\mathbf{w})$. Step η **too small**: slow.

Gradient descent: the step size



Just right: a few steps to the minimum.

Gradient descent: the step size



Too large: jumps over the minimum, may **diverge**.

Gradient descent with an adaptive step

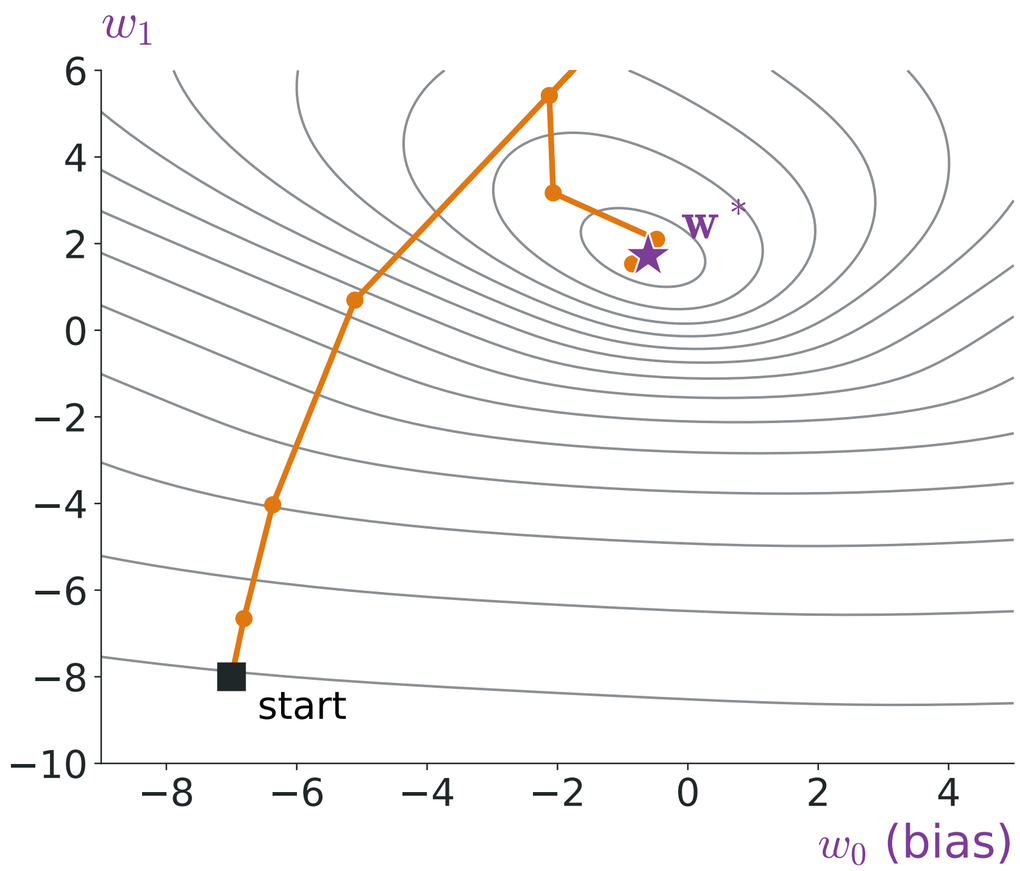
```
w = w_init
step_size = 1.0
E, g = compute_E_and_gradient(X, k, w)
while not TERMINATION_CONDITION:
    E_new, g_new = compute_E_and_gradient(X, k, w - step_size
    if E_new < E:        # success: accept, larger step
        w -= step_size * g
        E, g = E_new, g_new
        step_size *= 2
    else:                # failure: smaller step
        step_size /= 2
return w
```

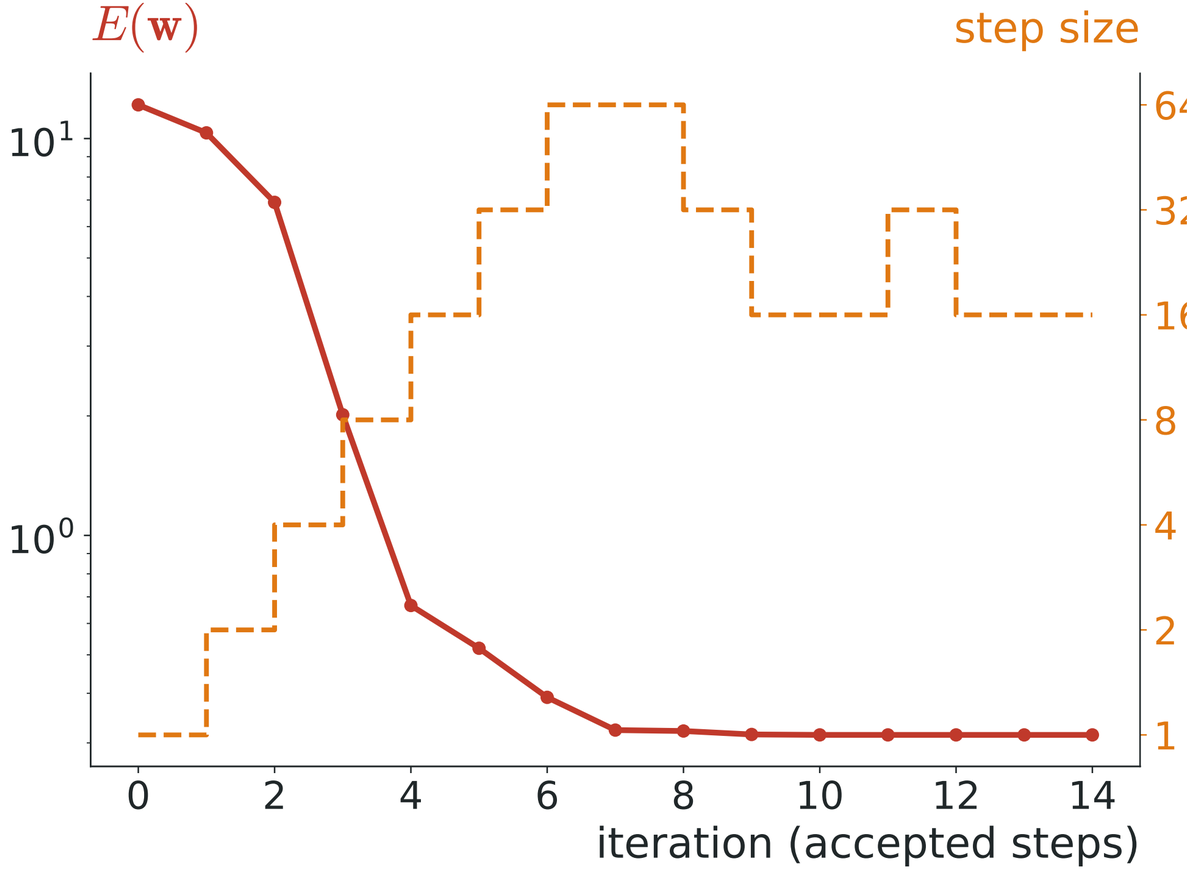
- accept the step only if E **decreases**
- success: **double** the step
- failure: **halve** it
- stop: $\|\mathbf{w}_{\text{new}} - \mathbf{w}_{\text{prev}}\| \leq \varepsilon$



The gradient is normalized by $\frac{1}{N}$: the step size does not depend on the number of samples.

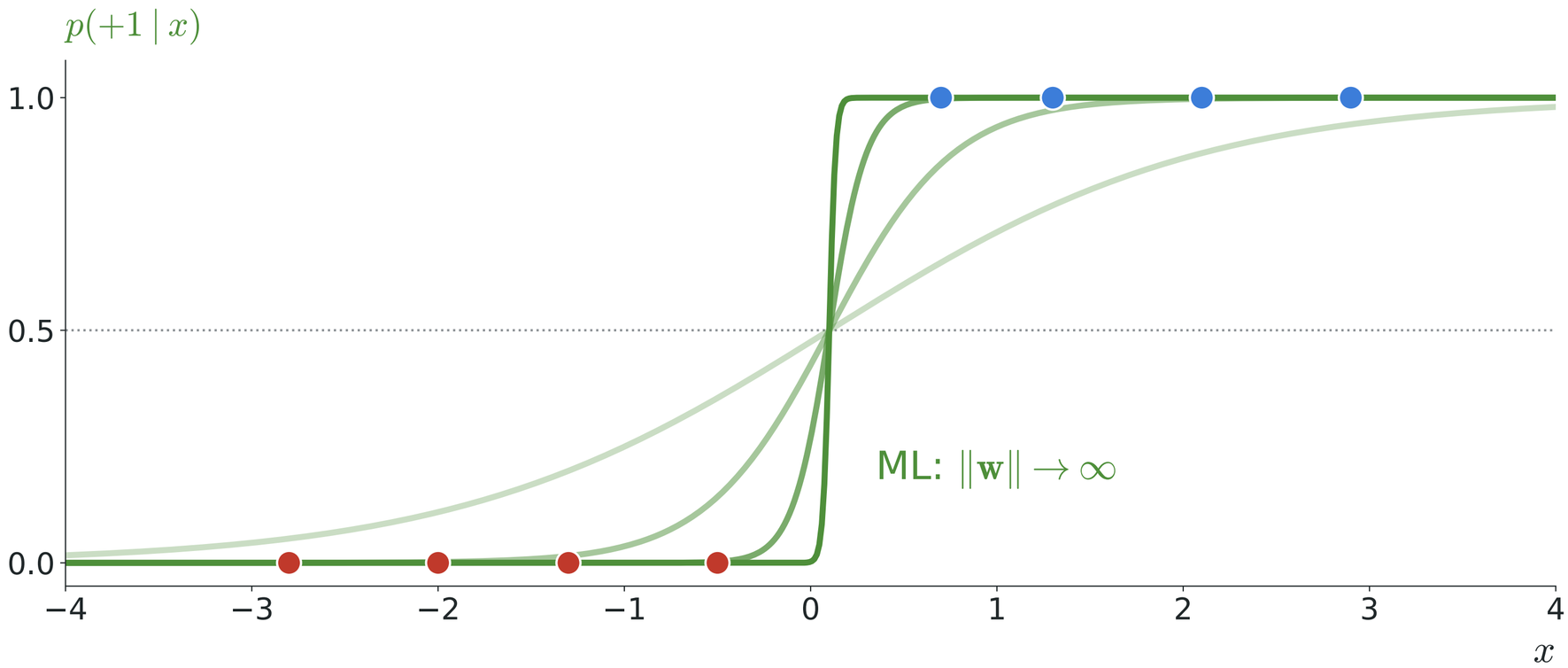
Gradient descent in action





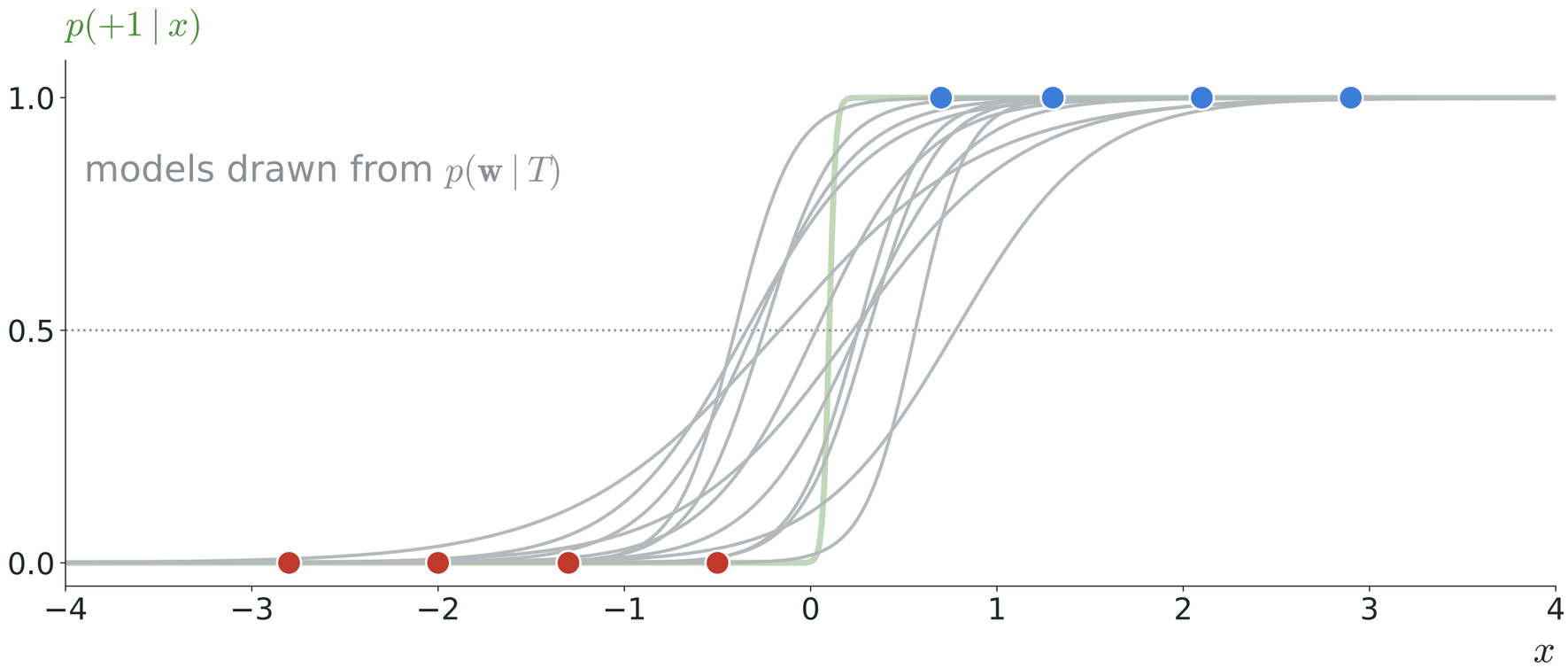
The 1D data from the interactive slide, start at $\mathbf{w} = (-7, -8)$ as in the assignment.

Separable data: ML is overconfident



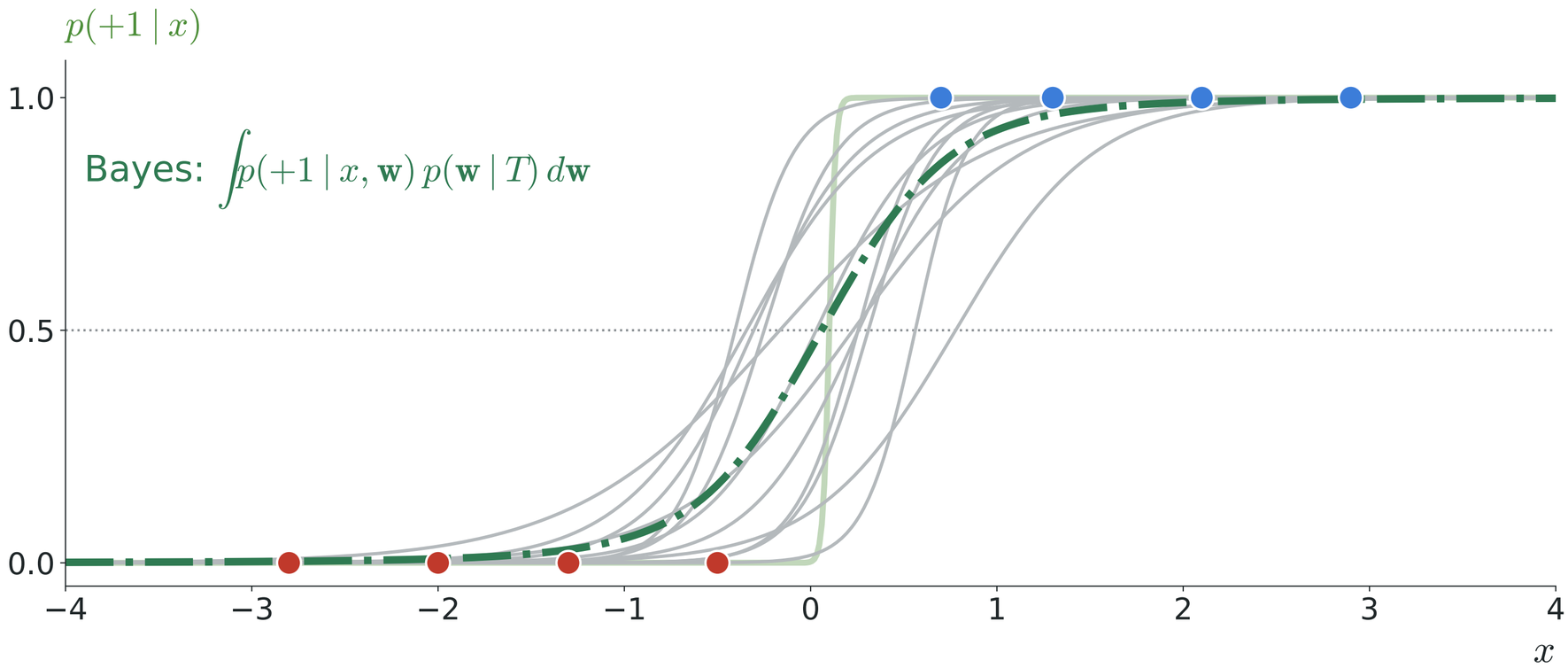
Separable data: every step of gradient descent makes $\|\mathbf{w}\|$ larger, $E \to 0$. The limit is a **step**.

Separable data: ML is overconfident



With a prior $p(\mathbf{w})$: the posterior $p(\mathbf{w} \mid T)$, models drawn from it.

Separable data: ML is overconfident



Bayes averages them (as in Lab 5): **uncertain** between the classes.

Generative vs discriminative

When is the log odds linear?

Two normal classes, the **same** Σ:

$$a(x) = \ln \frac{p(x \mid 1)\, p(1)}{p(x \mid 2)\, p(2)}$$

The terms $x^\top \Sigma^{-1} x$ **cancel**:

$$a(x) = \underbrace{(\mu_1 - \mu_2)^\top \Sigma^{-1}}_{\mathbf{w}_1^\top} x + w_0$$

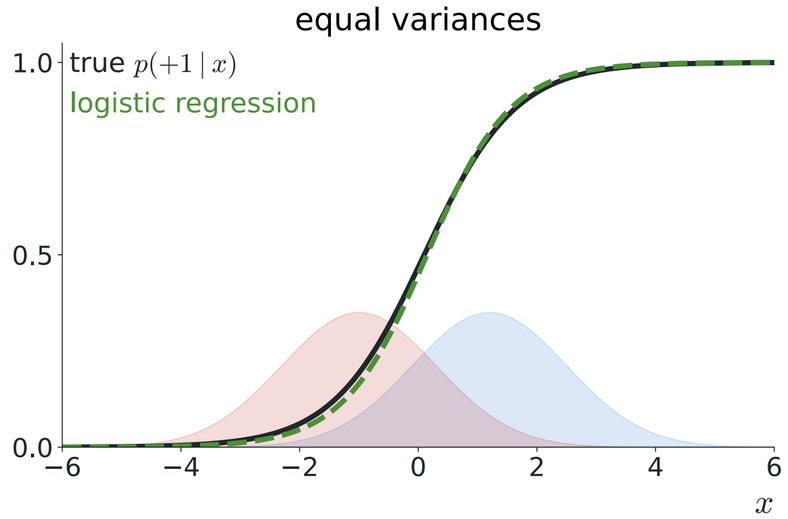
When is the log odds linear?

Two normal classes, the **same** Σ:

$$a(x) = \ln \frac{p(x \mid 1)\, p(1)}{p(x \mid 2)\, p(2)}$$

The terms $x^\top \Sigma^{-1} x$ **cancel**:

$$a(x) = \underbrace{(\mu_1 - \mu_2)^\top \Sigma^{-1}}_{\mathbf{w}_1^\top} x + w_0$$



Equal variances: logistic regression is the **exact** model.

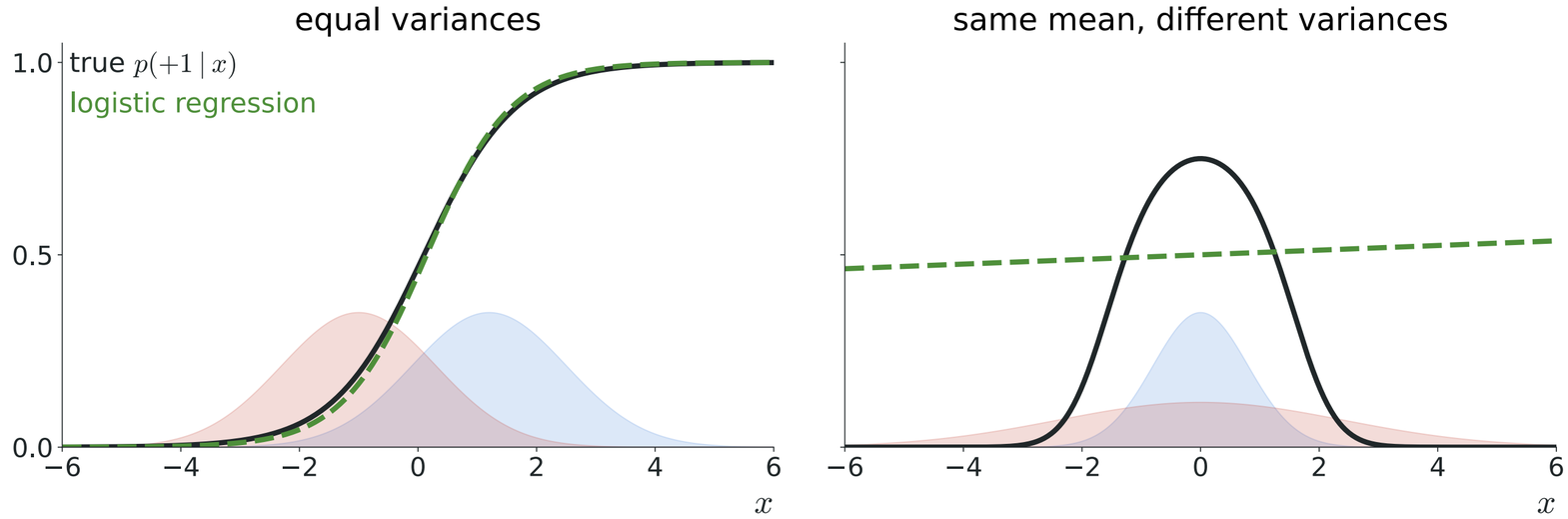
When is the log odds linear?

Two normal classes, the **same** Σ:

$$a(x) = \ln \frac{p(x \mid 1)\, p(1)}{p(x \mid 2)\, p(2)}$$

The terms $x^\top \Sigma^{-1} x$ **cancel**:

$$a(x) = \underbrace{(\mu_1 - \mu_2)^\top \Sigma^{-1}}_{\mathbf{w}_1^\top} x + w_0$$



Same mean, different variances: a **quadratic** log odds, logistic regression **fails**.

Number of parameters

Two classes, $\boldsymbol{x} \in \mathbb{R}^D$:

- **generative**: two normal densities with a **shared** covariance, priors
- **logistic regression**: $\mathbf{w}$



1. How many parameters does each model have?
2. $D = 2$? $D = 784$ (MNIST, 28×28)?

More problems like this: exercise book on CourseWare.

Derivation: number of parameters

generative: $\underbrace{2D}_{\mu_1, \mu_2} + \underbrace{\frac{D(D+1)}{2}}_{\Sigma} + \underbrace{1}_{p(1)} = O(D^2)$

logistic regression: $\underbrace{D+1}_{\mathbf{w}} = O(D)$

D	generative	logistic regression
2	8	3
784	309 289	785



Fewer parameters: fewer data needed. But if the generative model is **right**, it uses the data better.

Interpretability

$$a(x) = w_0 + \sum_{j=1}^{D} w_j x_j$$

- $|w_j|$ large: feature x_j is **important**
- sign of w_j: towards which class
- x_j up by 1: the odds multiplied by e^{w_j}

feature x_j	w_j
body temperature	2.1
cough	1.4
vaccinated	-1.8
shoe size	0.02

A made-up example: flu ($+1$) or not (-1).

> **!** Compare $|w_j|$ only for **normalized** features (same scale).

Beyond two classes and lines

More classes: softmax

One $\mathbf{w}_k$ per class:

$$p(k \mid x) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{m=1}^{K} e^{\mathbf{w}_m^\top \mathbf{x}}}$$

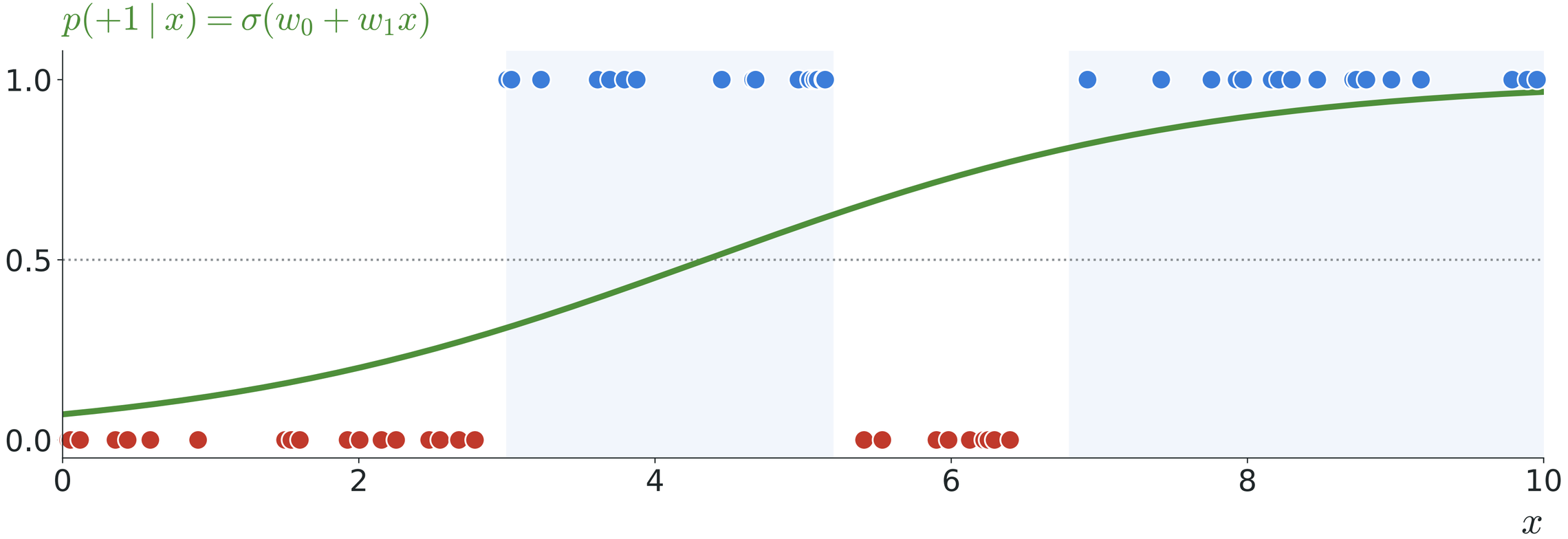
? Show that for $K = 2$ it is the sigmoid.

$$p(1 \mid x) = \sigma\left((\mathbf{w}_1 - \mathbf{w}_2)^\top \mathbf{x}\right)$$

$\mathbf{w}_k^\top \mathbf{x}$	$e^{\mathbf{w}_k^\top \mathbf{x}}$	$p(k \mid x)$
1.3	3.7	0.02
5.1	164.0	**0.90**
2.2	9.0	0.05
0.7	2.0	0.01
1.1	3.0	0.02

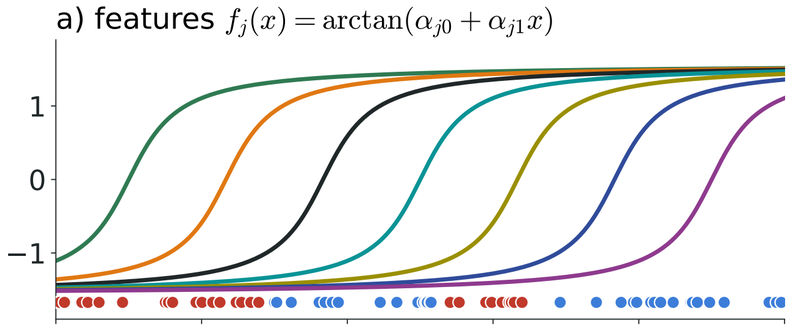
Exponentiate, normalize to sum 1.

Only a linear boundary?



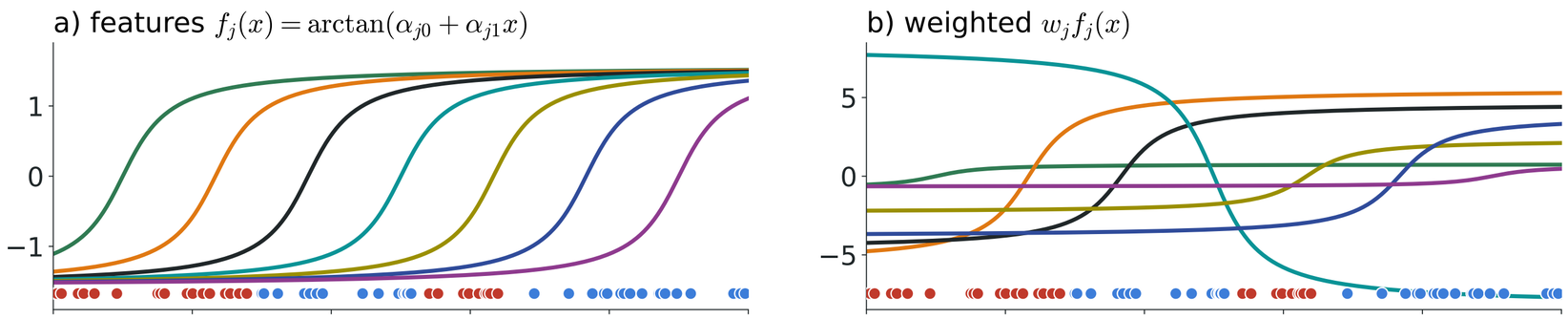
? The classes alternate. What can we do with one sigmoid?

Dimensionality lifting



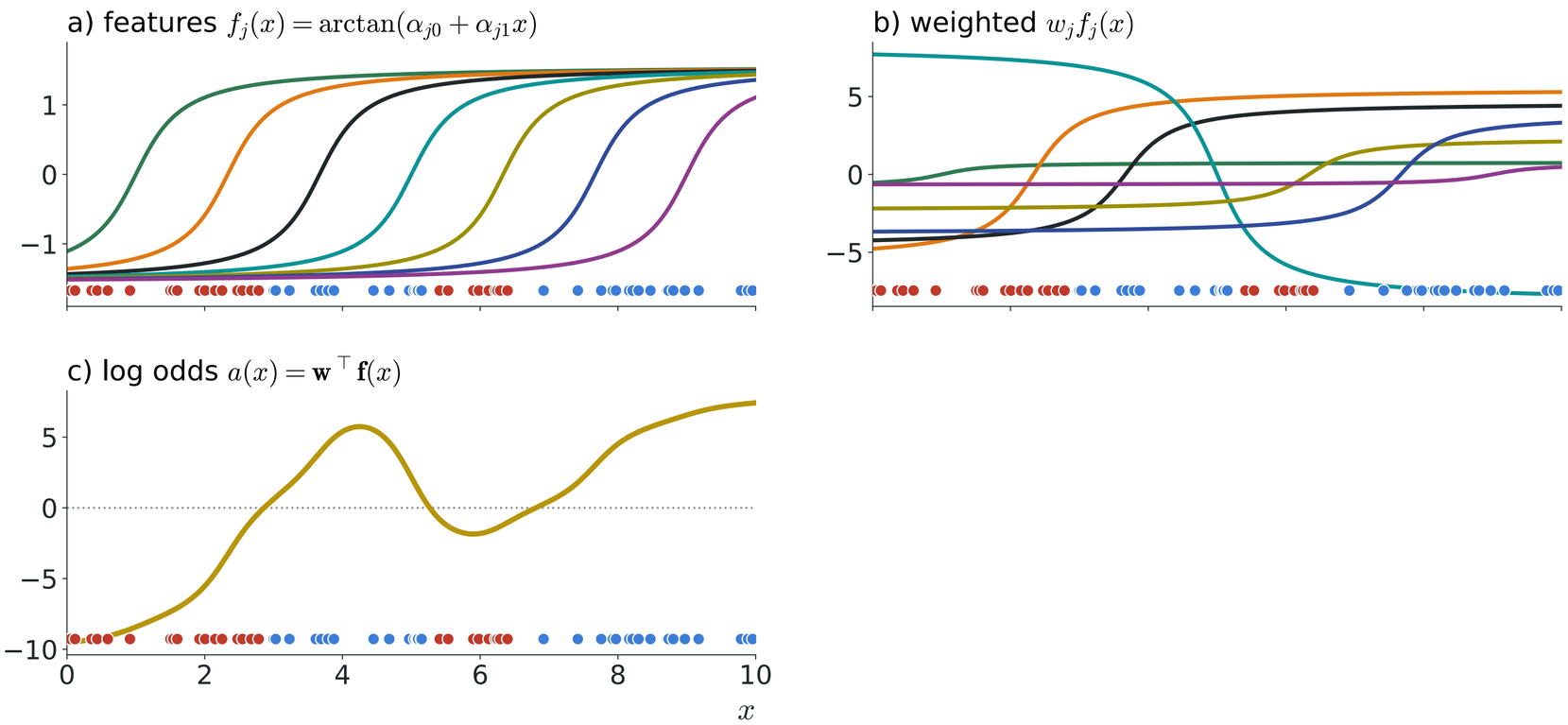
Fixed features $\mathbf{f}(x) = (f_1(x), \ldots, f_7(x)) \in \mathbb{R}^7$: lift x into 7 dimensions.

Dimensionality lifting



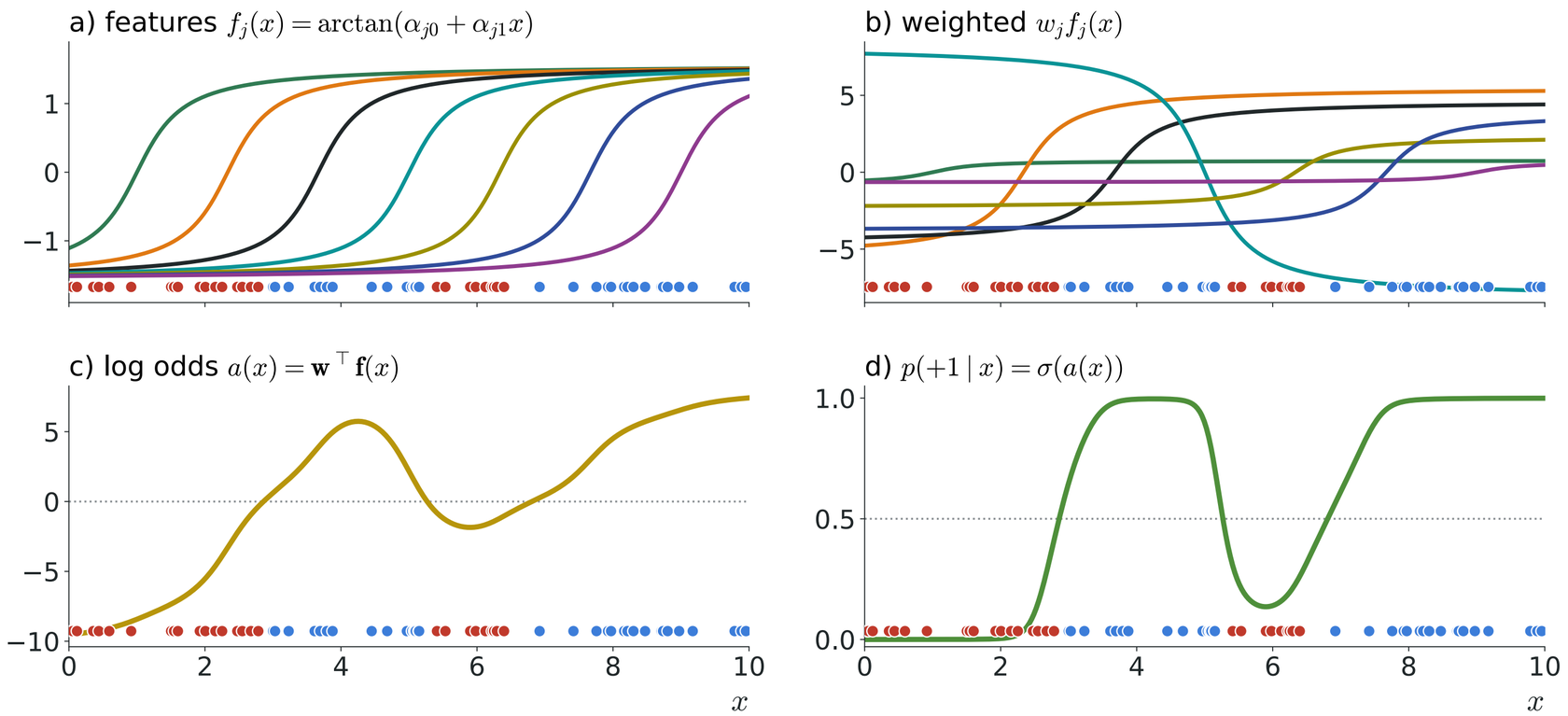
Logistic regression in $\mathbb{R}^7$ learns the weights w_j.

Dimensionality lifting



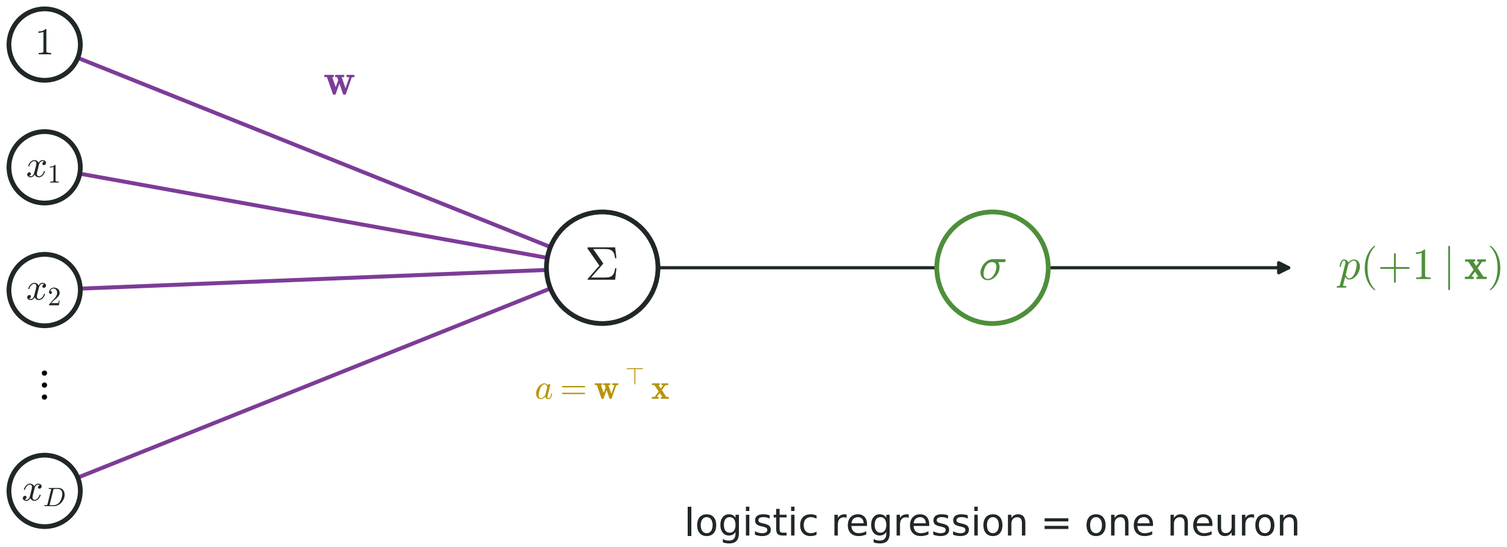
$a(x) = \mathbf{w}^\top \mathbf{f}(x)$: **linear in $\mathbf{f}$**, nonlinear in x.

Dimensionality lifting



Three thresholds in x: the **bonus task** (after Prince, 2012).

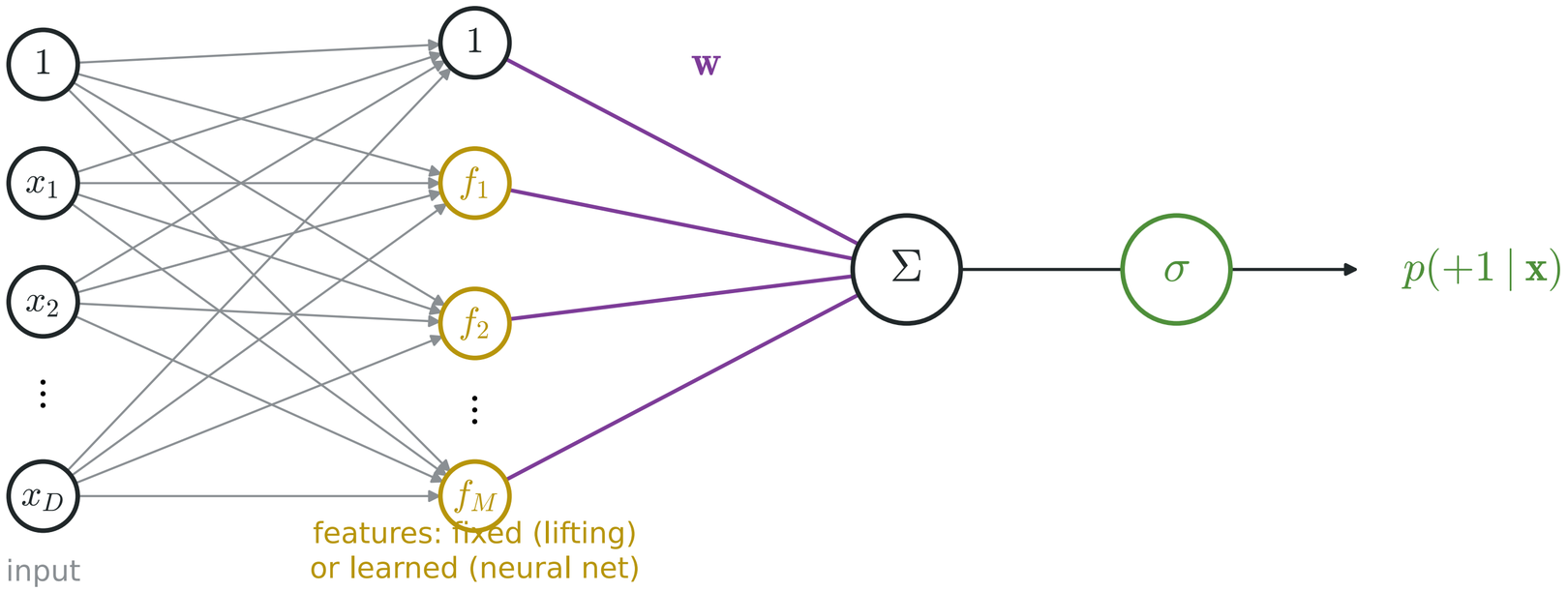
Towards neural networks



logistic regression = one neuron

Logistic regression: a weighted sum and a sigmoid, **one neuron**.

Towards neural networks



Features in a **hidden layer**: fixed (lifting) or **learned** (neural network, Lab 12).

High dimensions, many samples

Large D (e.g. 10^9)

- slow evaluation
- slow learning
- **GPUs**

Large N

- full gradient: slow
- **SGD**: gradient from a random **mini-batch**

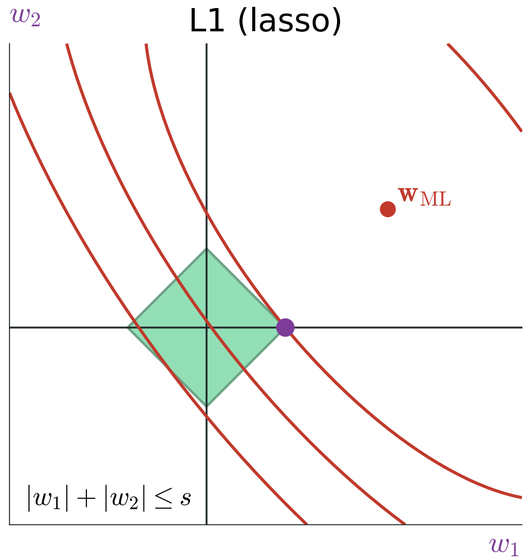
Few good features

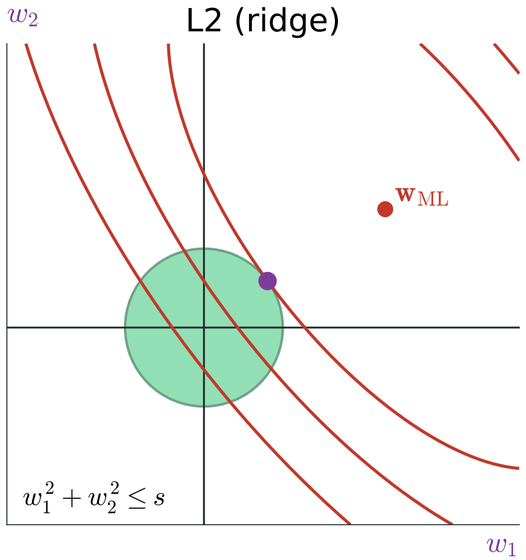
- **incremental**: add features one at a time
- $a(x) = \sum_{j=1}^{M} w_j f_j(x)$, $M \ll D$
- **AdaBoost** (Lab 10)

Regularization

$$E_\lambda(\mathbf{w}) = E(\mathbf{w}) + \lambda\|\mathbf{w}\|_2^2$$

- **L2** (ridge): MAP, a **normal prior** on $\mathbf{w}$ (Lab 5)
- **L1** (lasso), $\lambda\|\mathbf{w}\|_1$: a **sparse** $\mathbf{w}$
- **weight decay** in neural nets

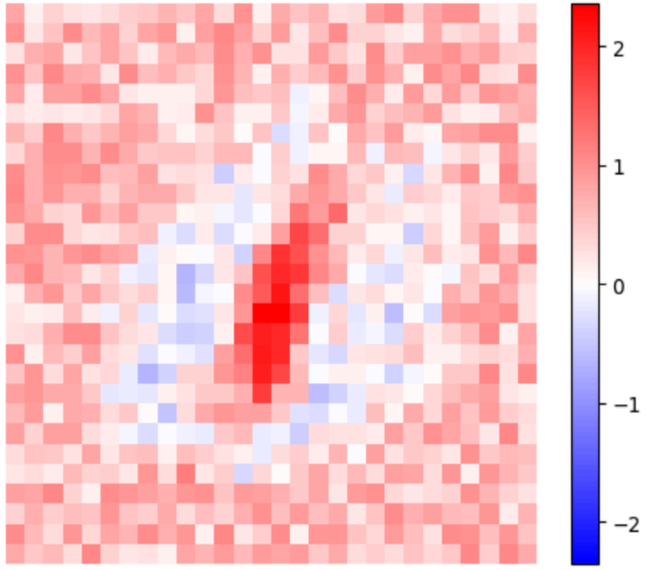




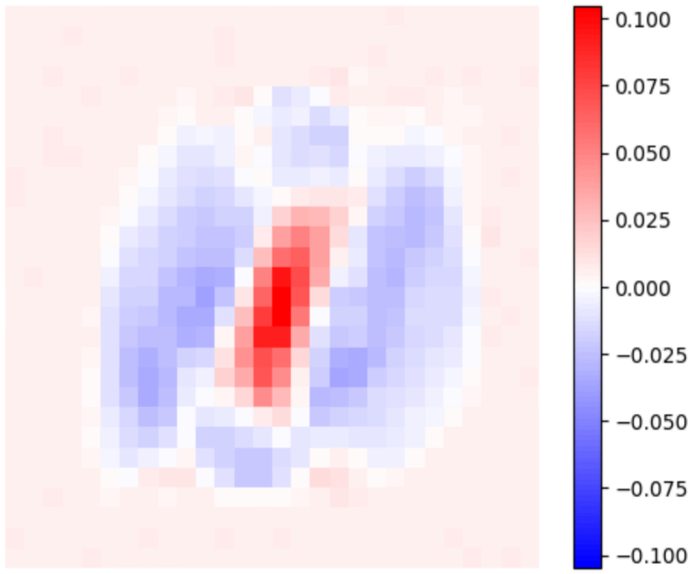
> **!** Separable data: $\|\mathbf{w}\|$ stays **finite**.

Regularization on MNIST: the weights

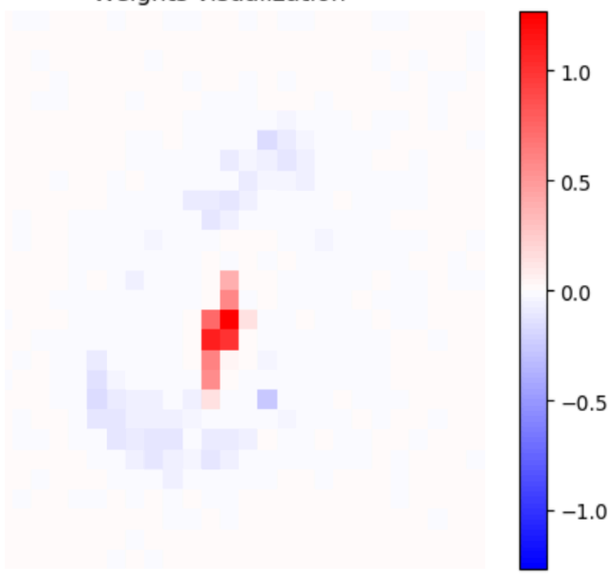
no regularization



L2



L1





How to choose λ, the features, the model? **Validation** (cross-validation from Lab 4).

One weight per pixel (784), from a solution of the assignment in previous years.

The assignment

Assignment: logistic regression

Part 1: letters A and C

- 1D feature: left minus right
- loss, gradient, descent
- test error about **7 %**

Part 2: MNIST

- pixels: $D = 784$
- the same functions
- the **weights** as an image

Bonus: lifting

- $\arctan$ features (Prince)
- nonlinear boundary in 1D
- labels ± 1, not 0, 1

! Add the **1** for the bias, normalize the test data with the **training** mean and variance, and divide the gradient by N.

Take-away for the lab

- ☑ **Discriminative**: model $p(k \mid x)$ directly, no $p(x \mid k)$
- ☑ Linear **log odds** $\iff p(k \mid x) = \sigma(k\,\mathbf{w}^\top \mathbf{x})$, a linear boundary
- ☑ ML $=$ min of the **cross-entropy**: convex, solved by **gradient descent**
- ☑ Separable data: $\|\mathbf{w}\| \to \infty$, use a prior (**regularization**) or Bayes
- ☑ **Softmax** for more classes, **lifting** for nonlinear boundaries: towards neural networks


"That's all Folks!"